Low-Level Design (LLD)

1 API Endpoint Definitions

Base URL: `http://localhost:8000`
All request/response bodies are `application/json`.

1.1 GET `/health`

Purpose: Liveness probe — confirms the process is running.
Auth: None

Response 200:

```
{ "status": "ok" }
```

1.2 GET `/ready`

Purpose: Readiness probe — confirms model is loaded and accepting traffic.
Auth: None

Response 200:

```
{ "status": "ready" }
```

Response 503:

```
{ "detail": "Model not yet loaded" }
```

1.3 POST `/predict`

Purpose: Predict sentiment for a single Amazon review.
Auth: None

Request Body:

```
{
  "review": "string (1-5000 chars, required)"
}
```

Response 200:

```
{
  "review": "string -- echoed input",
  "sentiment": "positive | negative",
  "label": "integer -- 1 (positive) or 0 (negative)",
  "confidence": "float -- sigmoid output probability [0.0, 1.0]",
  "latency_ms": "float -- end-to-end inference time in milliseconds",
  "threshold": "float -- optimal threshold used for classification"
}
```

Response 422 (Validation Error):

```
{
  "detail": [
    {
      "loc": ["body", "review"],
      "msg": "field required",
      "type": "value_error.missing"
    }
  ]
}
```

Response 503:

```
{ "detail": "Model not ready" }
```

Example curl:

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"review": "Absolutely love this product!"}'
```

Example response:

```
{
  "review": "Absolutely love this product!",
  "sentiment": "positive",
  "label": 1,
  "confidence": 0.9712,
  "latency_ms": 18.4,
  "threshold": 0.4879
}
```

1.4 POST `/predict/batch`

Purpose: Predict sentiment for multiple reviews in one call.
Auth: None
Constraints: Array length 1–50; each string 1–5000 chars.

Request Body:

```
{
  "reviews": ["string", "string", "..."]
}
```

Response 200:

```
{
  "predictions": [
    {
      "review": "string",
      "sentiment": "positive | negative",
      "label": 0,
```

```
      "confidence": 0.1234,
      "latency_ms": 35.2,
      "threshold": 0.4879
    }
  ],
  "total": 2,
  "latency_ms": 35.2
}
```

Response 422: Same structure as `/predict` validation error.

1.5 GET `/drift`

Purpose: Returns current data drift detection status based on KS test.
Auth: None

Response 200:

```
{
  "drift_detected": false,
  "ks_statistic": 0.3421,
  "p_value": 0.0012,
  "window_size": 500,
  "window_mean": 12.4,
  "baseline_mean": 37.08,
  "mean_shift": -24.68,
  "message": "DRIFT DETECTED (p=0.0012 < 0.05)"
}
```

Field definitions:

Table 1: `/drift` Response Field Definitions

Field	Type	Description
`drift_detected`	bool	True if KS test p-value < 0.05
`ks_statistic`	float—null	KS test statistic (0–1, higher = more different)
`p_value`	float—null	Probability distributions are the same. < 0.05 = drift
`window_size`	int	Current number of samples in rolling window
`window_mean`	float—null	Mean word count of current window
`baseline_mean`	float—null	Mean word count from training data
`mean_shift`	float—null	Difference: `window_mean` − `baseline_mean`
`message`	string	Human-readable status description

Note: Returns `null` for statistical fields until window reaches 500 samples.

1.6 GET `/metrics`

Purpose: Expose Prometheus metrics for scraping.
Auth: None
Content-Type: `text/plain; version=0.0.4`

Key custom metrics:

```
# Prediction counters
sentiment_predictions_total{sentiment="positive|negative"} <count>

# Latency
sentiment_prediction_latency_seconds_bucket{le="0.1"} <count>
sentiment_prediction_latency_seconds_sum <seconds>
sentiment_prediction_latency_seconds_count <count>

# Ratio
sentiment_positive_ratio <0.0-1.0>

# Input length distribution
sentiment_input_length_words_bucket{le="50"} <count>

# Model status
sentiment_model_loaded <0|1>

# Drift metrics
sentiment_drift_detected <0|1>
sentiment_drift_ks_statistic <0.0-1.0>
sentiment_drift_p_value <0.0-1.0>
sentiment_drift_mean_shift <float>
sentiment_drift_window_mean <float>
```

2 Request/Response Flow Diagram

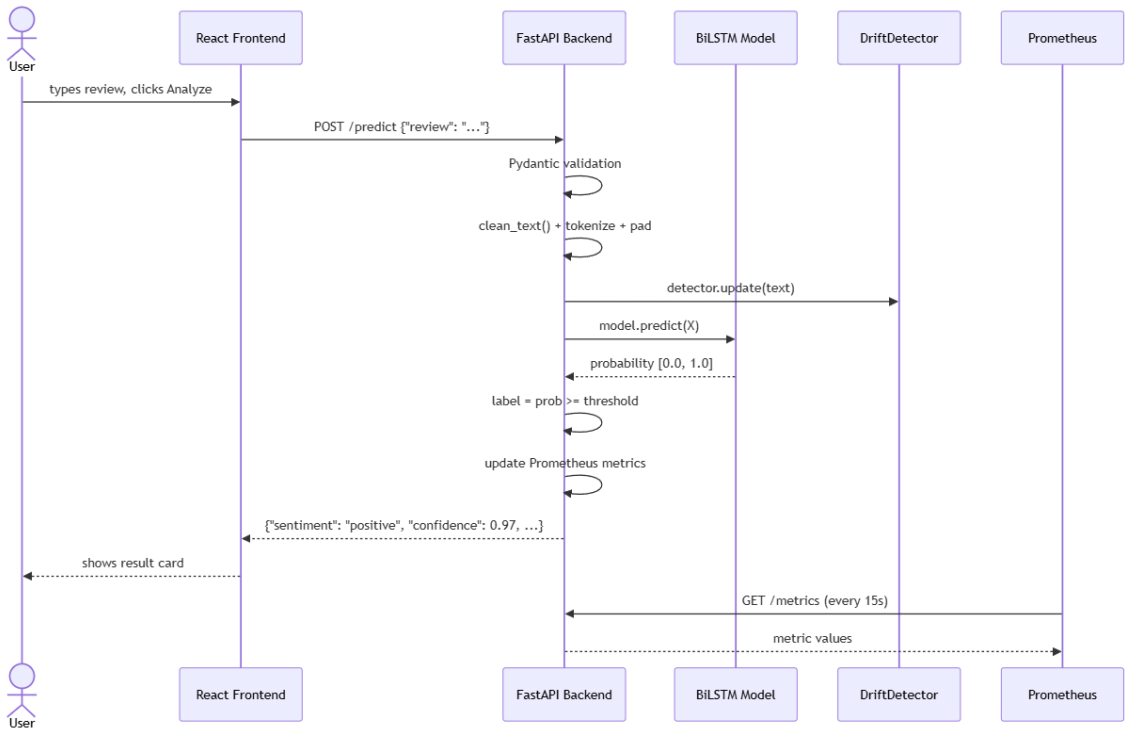

Figure 1: Request/Response Sequence Diagram (User → Frontend → FastAPI → BiLSTM → DriftDetector → Prometheus)

3 Module-Level Specifications

3.1 `src/data/ingest.py`

Table 2: `ingest.py` Function Signatures

Function	Signature	Input	Output	Raises
`validate_schema`	`(df: DataFrame) -> None`	DataFrame	None	`ValueError` if columns missing or invalid sentiment values
`log_statistics`	`(df: DataFrame) -> dict`	DataFrame	stats dict	—
`ingest`	`(cfg: dict) -> DataFrame`	config dict	cleaned DataFrame	`FileNotFoundError` if raw CSV missing

Key validations:

- Required columns: `review`, `sentiment`
- Valid sentiment values: {`0, 1`}
- Drops: `NaN` values, duplicate rows

3.2 `src/data/preprocess.py`

Table 3: `preprocess.py` Function Signatures

Function	Signature	Input	Output
`clean_text`	`(text: str) -> str`	raw string	cleaned string
`preprocess_text`	`(text: str) -> str`	raw string	clean lemmatized string
`compute_baseline_stats`	`(texts: list[str]) -> dict`	list of strings	stats dict
`SimpleTokenizer.fit_on_texts`	`(texts: list[str]) -> None`	list of strings	None (mutates self)
`SimpleTokenizer.texts_to_sequences`	`(texts: list[str]) -> list[list[int]]`	list of strings	list of integer sequences
`SimpleTokenizer.to_json`	`() -> str`	—	JSON string
`pad_sequences`	`(seqs, maxlen, padding, truncating) -> ndarray`	list of sequences	(N, maxlen) int32 array
`preprocess`	`(cfg: dict) -> None`	config dict	None (writes files)

3.3 `src/features/featurize.py`

Table 4: `featurize.py` Function Signatures

Function	Signature	Input	Output	Notes
`featurize`	`(cfg: dict) -> None`	config dict	None (writes files)	Loads CSVs → fits tokenizer → pads → saves numpy arrays

Outputs written:

- `data/processed/X_train.npy` — shape: (70556, 128), dtype: int32
- `data/processed/X_val.npy` — shape: (10080, 128), dtype: int32
- `data/processed/X_test.npy` — shape: (20160, 128), dtype: int32

- `data/processed/y_train.npy` — shape: (70556,), dtype: int64
- `data/processed/y_val.npy` — shape: (10080,), dtype: int64
- `data/processed/y_test.npy` — shape: (20160,), dtype: int64
- `data/processed/tokenizer.json` — SimpleTokenizer JSON
- `data/processed/tokenizer.pkl` — pickle for backward compatibility
- `data/processed/feature_meta.json` — vocab_size, max_sequence_length, split sizes

3.4 `src/models/train.py`

Table 5: `train.py` Function Signatures

Function	Signature	Input	Output	Notes
`build_model`	`(vocab_size: int, cfg: dict) -> keras.Model`	vocab size + config	compiled Keras model	BiLSTM architecture
`train`	`(cfg: dict) -> None`	config dict	None (writes files + logs MLflow)	Full training loop

Model hyperparameters (from `configs/config.yaml`):

Table 6: Model Hyperparameters

Parameter	Value	Rationale
`embedding_dim`	128	Rich word vector space for large vocabulary
`lstm_units`	128	Sufficient capacity without overfitting
`dense_units`	64	Smaller head reduces overfitting after LSTM
`dropout`	0.3	Standard regularization on LSTM inputs
`recurrent_dropout`	0.2	Regularizes recurrent connections specifically
`batch_size`	128	Stable gradients, efficient GPU utilization
`max_sequence_length`	128	Covers 95%+ of reviews without truncation
`optimizer`	adam	Adaptive learning rate, standard for NLP
`early_stopping_patience`	3	Stops quickly when `val_loss` plateaus

3.5 `src/models/evaluate.py`

Table 7: `evaluate.py` Function Signatures

Function	Signature	Input	Output	Notes
`evaluate`	`() -> dict`	—	error analysis report dict	Loads model + test set, computes error analysis, saves JSON

Error analysis outputs:

- Total errors and error rate
- False positives and false negatives counts
- High confidence errors (confidence > 0.8)
- Accuracy by review length bin (0–10, 10–20, 20–40, 40–80, 80+ words)
- 5 worst false positive and false negative examples

3.6 `src/api/main.py`

Table 8: `main.py` Components

Component	Type	Description
`app`	`FastAPI`	Main application with CORS + Prometheus instrumentation
`lifespan`	async context manager	Loads model, tokenizer, threshold, drift detector on startup
`_state`	`dict`	In-process state: model, tokenizer, ready flag, threshold, drift detector
`_preprocess_single`	`(text, tokenizer, max_len) -> np.ndarray`	Tokenizes and pads a single text
`_run_inference`	`(texts: list[str]) -> list[dict]`	Batch inference + metric updates + drift window update

3.7 src/api/model_loader.py

Table 9: `model_loader.py` Function Signatures

Function	Signature	Input	Output	Notes
`load_model_and_tokenizer`	`(cfg: dict) -> tuple`	config dict	(model, tokenizer)	Priority: local SavedModel → MLflow URI
`_load_model`	`(cfg: dict) -> keras.Model`	config dict	loaded model	Tries local path first, MLflow as fallback
`_load_tokenizer`	`(cfg: dict) -> SimpleTokenizer`	config dict	tokenizer	Priority: JSON → pickle

3.8 src/monitoring/drift_detector.py

Table 10: `drift_detector.py` Components

Component	Type	Description
`DriftDetector.__init__`	`(baseline_path, window_size=500, ks_threshold=0.05)`	Loads baseline stats, initializes rolling deque
`DriftDetector.update`	`(text: str) -> None`	Adds word count of incoming text to rolling window
`DriftDetector.check_drift`	`() -> dict`	Runs KS test if window full; returns drift report
`DriftDetector.drift_rate`	property → float	Ratio of drift checks that returned True

KS Test logic:

```
baseline_samples = Normal(mean=37.08, std=21.60, n=500) # synthetic from
    stored stats
window_array = deque of last 500 word counts
ks_statistic, p_value = scipy.stats.ks_2samp(window_array, baseline_samples)
drift_detected = p_value < 0.05
```

3.9 `src/monitoring/metrics.py`

Table 11: Prometheus Metrics Definitions

Metric	Type	Labels	Description
`sentiment_predictions_total`	Counter	`sentiment`	Total predictions by class
`sentiment_prediction_latency_seconds`	Histogram	—	Per-call inference latency
`sentiment_positive_ratio`	Gauge	—	Positive class ratio in last batch
`sentiment_input_length_words`	Histogram	—	Word count of incoming texts
`sentiment_model_loaded`	Gauge	—	1 when model is ready
`sentiment_drift_detected`	Gauge	—	1 when KS drift detected
`sentiment_drift_ks_statistic`	Gauge	—	Current KS test statistic
`sentiment_drift_p_value`	Gauge	—	Current KS test p-value
`sentiment_drift_mean_shift`	Gauge	—	`window_mean` − `baseline_mean`
`sentiment_drift_window_mean`	Gauge	—	Mean word count of current window

4 DVC Pipeline Specification

Figure 2: DVC Pipeline DAG: ingest → preprocess → featurize → train, with deps and outputs per stage

5 Airflow DAG Specification

DAG ID: `sentiment_data_pipeline`
Schedule: `@daily`
Catchup: False

Figure 3: Airflow DAG: `data_ingest` → `data_preprocess`

Table 12: Airflow Task Definitions

Task ID	Operator	Upstream	Description
`data_ingest`	PythonOperator	—	Runs `src.data.ingest.ingest()`
`data_preprocess`	PythonOperator	`data_ingest`	Runs `src.data.preprocess.preprocess()`

6 Docker Services Specification

Table 13: Docker Compose Service Configuration

Service	Build	Int. Port	Host Port	Key Volumes	Health Check
`backend`	`Dockerfile.backend`	8000	8000	`data/processed` (ro), `mlruns` (ro)	`GET /health`
`frontend`	`Dockerfile.frontend`	80	3002	—	nginx default
`mlflow`	`Dockerfile.mlflow`	5000	5000	`mlruns/`	—
`prometheus`	`prom/prometheus:2.47.0`	9090	9090	`prometheus.yml`, `alert_rules.yml`	—
`grafana`	`grafana/grafana:10.1.5`	3000	3003	`provisioning/`	—
`airflow`	`apache/airflow:2.7.3`	8080	8080	`airflow/dags/`, project root	`GET /health`

7 Configuration Reference (`configs/config.yaml`)

Table 14: Configuration Keys and Descriptions

Section	Key	Value	Description
`data`	`raw_file`	`data/raw/Amazon_Review.tsv`	Input data path
`data`	`processed_dir`	`data/processed`	Output artifacts directory
`preprocessing`	`max_sequence_length`	128	Token sequence length
`preprocessing`	`test_size`	0.20	Test split fraction
`preprocessing`	`val_size`	0.10	Validation split fraction
`preprocessing`	`random_state`	42	Seed for all random operations
`model`	`embedding_dim`	128	Embedding vector dimensions
`model`	`lstm_units`	128	BiLSTM units per direction

Continued on next page

Section	Key	Value	Description
model	dense_units	64	Hidden dense layer units
model	dropout	0.3	LSTM input dropout
model	recurrent_dropout	0.2	LSTM recurrent dropout
model	batch_size	128	Training batch size
model	epochs	15	Maximum training epochs
model	early_stopping_patience	3	Epochs to wait before stopping
mlflow	experiment_name	amazon-sentiment-bilstm	MLflow experiment name
mlflow	registered_model_name	SentimentBiLSTM	Model registry name
api	host	0.0.0.0	API bind address
api	port	8000	API port
monitoring	drift_threshold	0.05	KS test p-value threshold

8 Data Schemas

8.1 Raw Data Schema

Table 15: Raw Data Schema

Column	Type	Valid Values	Description
review	string	Any text	Amazon product review text
sentiment	int	0 or 1	0 = negative, 1 = positive

8.2 Processed Artifacts Schema

Table 16: Processed Artifacts Schema

File	Shape	dtype	Description
X_train.npy	(70556, 128)	int32	Padded training token sequences
X_val.npy	(10080, 128)	int32	Padded validation sequences
X_test.npy	(20160, 128)	int32	Padded test sequences
y_train.npy	(70556,)	int64	Binary training labels
y_val.npy	(10080,)	int64	Binary validation labels
y_test.npy	(20160,)	int64	Binary test labels
tokenizer.json	—	JSON	SimpleTokenizer word_index
baseline_stats.json	—	JSON	Drift detection baseline
optimal_threshold.json	—	JSON	{"threshold": 0.4879}
train_metrics.json	—	JSON	Final test set metrics
feature_meta.json	—	JSON	vocab_size, split sizes

8.3 baseline_stats.json Schema

```
{
  "mean_length": 37.088,
  "std_length": 21.595,
  "min_length": 1,
  "max_length": 161,
  "vocab_size": 105474,
  "total_samples": 70556
}
```

8.4 train_metrics.json Schema

```
{
  "test_accuracy": 0.8576,
  "test_f1": 0.8605,
  "test_precision": 0.8450,
  "test_recall": 0.8767,
  "test_roc_auc": 0.9360
}
```